NORDIC MACHINE INTELLIGENCE

Nordic Machine Intelligence, vol. 02, pp. 16–27, 2022
Received 13 04 2022 / published 26 11 2022
https://doi.org/10.5617/nmi.9633

A scalable, hybrid genetic algorithm for continuous multimodal optimization in moderate dimensions

K. Johannsen[1,2], N. Goris[1], B. Jensen[1], and J. Tjiputra[1]

[1]NORCE Norwegian Research Centre AS, Nygårdsgaten 112, 5008 Bergen, Norway
[2]E-mail any correspondence to: kljo@norceresearch.no

Abstract
Optimization problems can be found in many areas of science and technology. Not only the global optimum, but also a (large) number of near-optima are often of interest. This gives rise to what are referred to as multimodal optimization problems. In most cases, the number and quality of the optima are unknown and assumptions cannot be made about the objective functions. In this paper, we focus on continuous, unconstrained optimization in moderately high-dimensional continuous spaces ($d \lesssim 10$). We present a scalable algorithm, which performs well for general objective functions (non-convex, discontinuous). It is based on two well-established algorithms (CMA-ES, deterministic crowding). Novel elements of the algorithm include the detection of seed points for local searches and collision avoidance, both based on nearest neighbor search, and a strategy for semi-sequential optimization to realize scalability. The performance of the proposed algorithm is numerically evaluated on the CEC2013, CEC2015, and CEC2019 benchmark suites comprising $1-30$ dimensional functions, and a 9 dimensional real-world problem from constraint optimization in climate research. The algorithm generally performs well on the CEC benchmarks. In the case of the climate-related problem, the algorithm is able to find a high number (> 150) of optima of relevance to climate research. The proposed algorithm is easy to configure. Essentially three parameters need to be specified, for which good default values are available.

Keywords: CMA-ES, Continuous optimization; hybrid genetic algorithms; multimodal; niching

1. Introduction

Most real-world optimization problems have more than one optimal or near optimal solution. Such problems are called multimodal optimization (MMO) problems. They are of great importance in practice, for example for engineers to understand problems better or for decision-makers to make appropriate decisions. In most practical cases, the number and quality of optima or near-optima are not known. There is therefore a need for black-box algorithms, which do not require any a priori information about the optimization problem [1]. Furthermore, in order to avoid repeated numerical investigations, it is also highly desirable to have algorithms that do not require (complicated) configuration, and ideally none.

Many different strategies have been developed to solve multimodal problems. Among the most successful methods are evolutionary algorithms [2, 3]. Originally they are designed to locate single optima, and precautions must be taken to preserve diversity in the population in order to approximate multiple optima simultaneously. These techniques are commonly referred to as niching methods. Many of those methods have been developed, such as crowding [4, 5], fitness sharing [6], restricted tournament selection [7], or speciation [8]. These strategies have been modified and refined later, and new ones have been invented. The ideas have been developed for evolutionary algorithms (genetic algorithms, evolutionary strategies, differential evolution, etc.), but can, with modifications, also be applied to other population-based meta heuristics (ant colony optimization, particle swarm optimization, etc.) [9, 10]. To improve the normally rather slow convergence of evolutionary algorithms, they are often combined with specially designed core algorithms (local solvers), like hill-climbing algorithms, simulated annealing, the Nelder-Mead method, or CMA-ES. This combination is called hybridization, and here again different strategies,

such as sequential, Baldwinian, or Lamarckian hybridization are used [11, 12, 13, 14].

In this work, we use a deterministic crowding genetic algorithm and a CMA-ES local solver. The algorithms are combined by sequential hybridization, and embedded in a loop realizing a linearly increasing population (semi-sequential mode) [15]. The aim of our development is to design an easy to configure algorithm. We therefore choose the components of our algorithm such that good generic choices are possible for them.

The remainder of this paper is organized as follows. In Section 2, we present the overall design of the algorithm as well as its parts. For well-known parts, we discuss their configuration, and more details are provided in the case of new developments. In Section 3, we report on numerical experiments. We present the performance on the CEC2013 niching benchmark problems [16], the CEC2015 benchmark suite [17], the 100-Digit challenge from CEC2019 [18], and on an optimization problem from climate research (constraint optimization) [19]. The results are discussed in Section 4 and conclusions are drawn in Section 5. For the remainder of the paper, we will assume that the optimization problem is posed as a maximization problem and will consequently talk about maxima instead of optima, and the maximization problem instead of the optimization problem.

2. The proposed algorithm

In this work, we present an optimization algorithm for real-valued multimodal optimization problems in $d \geq 1$ dimensions, which requires the configuration of four parameters. Two of them configure an outer genetic algorithm, one configures the algorithm to generate seed-points for the core solver. For these three parameters good generic choices seem to be available. The fourth parameter is the maximum number of function evaluations, the stopping criterium.

Let $X \subset \mathbb{R}^d, d \geq 1$ be the real-valued search space and $f : X \to \mathbb{R}$ a real-valued objective function to be maximized. The solution to the multimodal maximization problem is to find

$$S = \{x \in X \mid x \text{ is a local maximum of } f\}. \qquad (1)$$

In many of the practical cases, the algorithm will only find a subset $\tilde{S} \subset S$ of all maxima. The algorithm presented here is invariant under strictly monotonous linear transformation of the objective function. Maxima can be expected to be detected in order of decreasing size of the corresponding domains of attraction, see Section 2.2.

2.1. Overall strategy

The proposed algorithm has a hybrid structure. To approximate niches, a deterministic crowding genetic algorithm [4] is used as the niching algorithm. After a fixed number of generations of this genetic algorithm, a nearest neighbor search [20] is employed to determine starting points and centroids as input for the local solver, a covariance matrix adaptation evolution strategy (CMA-ES) [21, 22]. An outer loop repeatedly calls the niching algorithm and successively increases the population of the niching algorithm. In this way, we realize semi-sequential behavior. We call the overall algorithm the *Scaling Hybrid Genetic Algorithm (SHGA)*, see Algorithm 1 below.

Algorithm 1: *Scaling Hybrid Genetic Algorithm (SHGA)*

Function: SHGA
Input: objective function f, max # function evaluations N, population increment n_p, # niching generations n_g, size of neighborhood n_{nb}
Output: set of maxima X

```
1  p ← ∅
2  while True do
3      //incrase population
4      p ← p ∪ [random population of size n_p]
5      //approximate niches
6      p ← Niching Algorithm(f, p, n_g)
7      if # fct evals > N then return (X)
8      //get seeds points for local solves
9      S ← Seeds(f, p, n_nb, X),  S = {(s_i, δ_i)}_{i=1,...|S|}
10     for k = 1, ... |S| do
11         //local solve
12         x ← Local Solver(f, s_k, δ_k)
13         if # fct evals > N then return (X)
14         X ← X ∪ {x}
15     end
16 end
```

In Algorithm 1, the outer loop (line 2) increases the size of the population at the beginning of each iteration by randomly generating n_p individuals (line 4). They are added to the previously existing individuals, which remain unchanged. Thereafter, the niching algorithm is executed for the constant number of n_g generations on the increased population (line 6). After that, potential new niches are identified by a nearest neighbor search-based algorithm as seed points $\{s_i\}$ with the associated radii $\{\delta_i\}$ of the niches (line 9). This algorithm is described below (Algorithm 2). Already determined maxima, stored in X, are used by the algorithm to exclude non-promising seed points. We call the algorithm used to detect new seed points (and the associated radii) the seeds algorithm. For each seed point the local solver is called (line 12) and the (local) maximum is added to the set of maximizers (line 14). The while loop is ended and the result is returned when the maximum number of function evaluations is exceeded (line 7 and 13).

The algorithm depends on a number of parameters. In addition to the internal configuration parameters of

the niching algorithm (such as mutation and crossover probabilities, etc.) and the local solver (tolerances etc.), Algorithm 1 requires four essential parameters to be specified: N, n_p, n_g and n_{nb}. N is the maximum number of function evaluations and serves as the stopping criterion. Typical values range from 10^4 for simple problems to 10^6 or more for complex problems. The value must be provided by the user. For the other three parameters, we can make generic choices. In practical applications it is often sufficient to specify only the stopping criterion (parameter N). The choices for the three parameters n_p, n_g and n_{nb} are discussed in section 2.5 below.

2.2. Niching algorithm

The niching algorithm is used to simultaneously approximate niches. To this end we use a deterministic crowding genetic algorithm [4]. In each generation, it performs the standard operations of parent selection, recombination, mutation, and survivor selection. As parents, all members of the population are chosen and mate with uniform probability (panmictic mixing). The recombination is performed with 30% probability using bounded simulated binary crossover with a crowding degree of $\eta = 10$. Afterwards, mutation is performed with 30% probability using bounded polynomial mutation with a crowding degree of $\eta = 10$. For details, see [23, 24]. At the end of each generation, deterministic crowding is used for the competition between the parent and the offspring population.

In Algorithm 1, the niching algorithm is executed for a fixed number of n_g generations. This leads to sufficient convergence of individuals towards different niches, where the number of individuals in the niches corresponds approximately linearly to the size of the niches. The number of individuals in the niches is not expected to depend significantly on the value of the associated maximum, see, e.g., [25, 26]. Regarding the choice of n_g, we refer to the discussion in Section 2.5.

2.3. Seeds algorithm

In Algorithm 1, line 9, the seeds algorithm is used to detect new niches. It returns a number of so-called seed points, one for each potential new niche. They are found as local maxima in the current population of the niching genetic algorithm. An approximate niche size is determined for each seed point. The algorithm is based on the identification of nearest neighbors and essentially depends on one parameter, the neighborhood size n_{nb}. The algorithm is given below as Algorithm 2.

The first loop of the algorithm (line 3) identifies individuals, which are maxima in a local neighborhood (line 7). The local neighborhood is formed by n_{nb} neighbors and the "mirrored" points $2p_k - p_{nb}$ (line 5 and 6). Note that this neighborhood generally consists of the central point under consideration (p_k) and an additional $2\,n_{nb} - 2$ points. The maximum distance from p_k to one of the points serves as an estimate of the associated niche size. In the second loop (line 14), the potential seed points are filtered. Points that lie too close to already known maxima (X) are excluded (line 16). The closeness criterion is related to the estimated niche size.

To keep the description simple, we have used a number of intuitive notations in Algorithm 2: in line 4, $p[k]$ refers to the k-th element of set p in an arbitrary enumeration, and in line 6, $(2p_k - p_{nb})$ denotes the set $\{2p_k - q \mid q \in p_{nb}\}$, in line 7 $f(p_{nb})$ denotes the set of function values of f for each point in p_{nb}. The expressions in line 15 and line 16 should be understood correspondingly. Note also that the meta-code given in Algorithm 2 is for illustration purposes. An efficient implementation in terms of the number of function evaluations would be implemented differently.

Algorithm 2: Seeds

Function: Seeds
Input: objective function f, population p, neighborhood size n_{nb}, maxima X
Output: set $S = \{(s_i, \delta_i)\}$ of seed points s_i with a niche radius of δ_i

```
1  //select potential seeds as local maxima
2  S_0 ← ∅
3  for k = 1, ... |p| do
4      p_k = p[k]
5      p_nb ← {n_nb nearest neighbors of p_k in the
         Euclidean norm} //this includes p_k itself
6      p_nb ← p_nb ∪ (2p_k − p_nb)
7      if min(f(p_nb)) == f(p_k) then
8          δ_k ← max({‖p_k − p_nb‖})
9          S_0 ← S_0 ∪ {(p_k, δ_k))}
10     end
11 end
12 //exclude seeds in neighborhood of known
     solutions
13 S_1 ← ∅
14 for k = 1, ... |S_0| do
15     (s_k, δ_k) = S_0[k]
16     if min(‖s_k − X‖) > δ_k then
17         S_1 ← S_1 ∪ {(s_k, δ_k))}
18     end
19 end
20 return (S_1)
```

2.4. Local solver

We use the covariance matrix adaptation evolution strategy (CMA-ES) as a local solver. The local solver for the niche identified by (s_i, δ_i) is initialized with the seed point s_i and a spheric centroid with a radius of $\delta_i/3$. We use the Python implementation by N. Hansen et. al. [22] with default configuration.

2.5. Critical parameters

In this section, we discuss the critical parameters n_p, n_g and n_{nb}, and how to choose them for good performance

of the overall algorithm. A summary of the parameters and the values is provided at the end of this section in Table 1.

Population increase n_p. The parameter n_p sets the initial population of the niching genetic algorithm and its further increment in every iteration of the `while`-loop in Algorithm 1, line 2. The choice of n_p should allow for a reasonable number of new maxima to be detected in each iteration of the `while`-loop. As a number of n_{nb} individuals is necessary to establish one seed point (see Algorithm 2, line 5), a small multiple of this number is a good choice. In our experience, a factor five or higher leads to good results; the exact choice is not critical. In this paper, we make the following generic choice

$$n_p = 10\, n_{nb}. \tag{2}$$

Number of generations n_g for the niching algorithm. The aim of the niching algorithm is to approximate niches. To this end a significant contraction of individuals towards the maximum of each niche needs to be accomplished. Numerical investigations have shown that 20 generations lead to a contraction of approximately $0.3 - 0.8$, depending on the problem, see Section 3. The contraction rate is measured in the statistical sense. With $n_g = 20$, Algorithm 2 yields good seed points in terms of their number compared to duplicate solutions obtained by the local solver. Experiments with $n_g \leq 10$ lead to rather poor seed points, $n_g \geq 40$ does not improve the quality of the seed points significantly enough to compensate for the additional effort. We therefore make the generic choice

$$n_g = 20. \tag{3}$$

Size of neighborhood n_{nb} in the seeds algorithm. The choice of n_{nb} is crucial to the performance of the entire algorithm. Too small a size will lead to too many seed points, initiating too many local solves, which often either do not converge or converge to already known maxima. A too large neighborhood size will lead to few seeds. This will cause few local searches and, correspondingly, to the detection of only a few maxima. To compensate for that, the population size of the niching genetic algorithm would need to increase much more, leading to an inefficient algorithm.

A smooth objective function f can be locally approximated by a quadratic function, which in turn yields information about the location of the local maximum (if it exists). It is therefore reasonable to relate the neighborhood size to the amount of information needed to approximate f by a quadratic polynomial. The minimum required number is therefore given by the degrees of freedom of a quadratic polynomial in d dimensions

$$n_{nb} = 1 + 1.5d + 0.5d^2, \tag{4}$$

which is our generic choice. For the detection, we add the "mirrored" points $2p_k - p_{nb}$ (Algorithm 2, line 6). This is to avoid generating seeds lying at the border of a cloud of individuals, probably located on the slope of a niche and potentially far away from its maximum.

Table 1: Generic choices of critical parameters of SHGA in dependence on the problem dimension d.

parameter	**value**	**description**
n_p	$10n_{nb}$	population size
n_g	20	# of generations
n_{nb}	$1 + 1.5d + 0.5d^2$	# of neighbors

3. Experiments

We evaluate the proposed algorithm on the CEC2013 niching benchmark suite [16], the CEC 2015 benchmark [17], the 100-Digit Challenge (CEC2019) [18], and a multimodal optimization problem arising from constraint optimization in climate research. For the CEC2013 niching benchmark suite, the locations of the absolute maxima are known, and we also investigate the contraction of the niching algorithm (crowding GA) for these problems, justifying the choice of (3).

It shall be noted that SHGA has been developed using the CEC2013 benchmark, so the algorithm might be biased towards this benchmark. The CEC2015, CEC2019 and the climate problem were not used in the development and with respect to these problems the algorithm is unbiased.

3.1. CEC 2013 niching benchmark suite

The CEC2013 niching benchmark suite consists of 20 problems with dimensions ranging from 1 to 20. The problems are listed in Table 2. They are to be solved within the given budget of function evaluations. For each problem, the solution procedure is called 50 times and the average peak ratio is reported, the peak ratio being the fraction of the global maxima detected. For details, see [16].

The performance of Algorithm 1 on the benchmark suite is shown in Table 3. The average peak ratio and an average contraction number (see below) is given for each of the benchmark problems. The performance of the algorithm (average peak ratio) is good for all problems, with the exception of the Vincent problems (problems 7 and 9) and the 10D and 20D problems (problems 18 - 20).

A comparison of the proposed algorithm with the algorithms from the most recent international competition on this benchmark suite (GECCO 2020) is shown in Table 4. We compare the results based on Scenario I. For details, see [27]. The proposed algorithm would rank number 5 compared to the other seven competing algorithms.

The poor performance of our algorithm on the Vincent

Table 2: Niching benchmark suite from the CEC2013 special session on multi-modal optimization [16]. For each problem function name, the problem dimension d, number of maxima #max (global/local), and the budget (maximum # function evaluations) are given.

#	function name	d	#max	budget
1	Five-Uneven-Peak Trap	1	2/3	50k
2	Equal Maxima	1	5/0	50k
3	Uneven Decreasing Max.	1	1/4	50k
4	Himmelblau	2	4/0	50k
5	Six-Hump Camel Back	2	2/2	50k
6	Shubert	2	18/many	200k
7	Vincent	2	36/0	200k
8	Shubert	3	81/many	400k
9	Vincent	3	216/0	400k
10	Modified Rastrigin	2	12/0	200k
11	Composition Function 1	2	6/many	200k
12	Composition Function 2	2	8/many	200k
13	Composition Function 3	2	6/many	200k
14	Composition Function 3	3	6/many	400k
15	Composition Function 4	3	8/many	400k
16	Composition Function 3	5	6/many	400k
17	Composition Function 4	5	8/many	400k
18	Composition Function 3	10	6/many	400k
19	Composition Function 4	10	8/many	400k
20	Composition Function 4	20	8/many	400k

Table 3: Performance of Algorithm 1 on the CEC2013 benchmark [16]. For each problem, the average peak ratio PR (fraction of global maxima detected) is given, the average # of function evaluations and the geometric average of the contraction number ξ, as defined by Algorithm 3. The averages are taken over 50 runs.

#	function name	avg. PR	avg. ξ
1	Five-Uneven-Peak	1.0000	0.3656
2	Equal Maxima	0.9960	0.5838
3	Uneven Decreasing Max.	1.0000	0.7087
4	Himmelblau	1.0000	0.6343
5	Six-Hump Camel Back	1.0000	0.5753
6	Shubert	0.9991	0.7178
7	Vincent	0.7450	0.6599
8	Shubert	0.9672	0.7830
9	Vincent	0.2931	0.6943
10	Modified Rastrigin	1.0000	0.6857
11	Composition Function 1	0.9940	0.7282
12	Composition Function 2	0.9975	0.7439
13	Composition Function 3	0.9673	0.7358
14	Composition Function 3	0.7033	0.7938
15	Composition Function 4	0.7400	0.7960
16	Composition Function 3	0.6667	0.8106
17	Composition Function 4	0.6975	0.8164
18	Composition Function 3	0.2300	0.8454
19	Composition Function 4	0.1450	0.8106
20	Composition Function 4	0.0000	0.8611

problems and on the higher dimensional problems stems from the small basins of attraction of a number of maxima.

Convergence of the niching algorithm. For the CEC-2013 benchmark suite, all global maxima are known. With this knowledge, we can define a contraction number $\xi \in [0, \infty[$ that measures the speed of convergence of the niching genetic algorithm. In essence, it describes the average movement of individuals in the population towards their nearest global maximum (in the Euclidean norm) over a number of generations, in our case n_g. A contraction number $\xi < 1$ corresponds to convergence, and a contraction number $\xi > 1$ to divergence. The contraction number is best described algorithmically; a pseudo code is given with Algorithm 3. As above, we use a simplified, intuitive notation to keep the description simple. The sets S, P_0, P_1 in Algorithm 3 have an order, where the order of S corresponds to the order of the partitioning subsets of P_0, P_1. We use indexes to denote access to the element in this order. Finally, the order within the partitioning subsets corresponds to the increasing distance to the corresponding elements in S.

Since we use clustering with respect to the Euclidean distance, the contraction number is strictly speaking only meaningful for multimodal problems with isotropic, equally sized basins of attraction. We measured contraction numbers for the benchmark problems, which vary roughly from 0.3 to 0.8. With a contraction of $\xi \leq 2/3$, good clustering around the associated maxima is achieved, sufficient for clustering-based seed detection. This is true for 5 of the benchmark problems. For another 13 benchmark problems, this level of contraction is reached after a second execution of n_g iterations, while, for the remaining 2, it is reached after a third execution of n_g iterations. Although this argument is rather simplistic, we believe that it captures the essence of the important feature of the niching algorithm. We consider this as the first explanation for why a generic choice of $n_g = 20$ leads to successful overall behavior of the proposed algorithm.

3.2. CEC 2015 benchmark suite

The CEC2015 niching benchmark suite consists of 15 types of problems (functions) with dimensions from 2 to 30. The functions are scalable, i.e. they are available for different dimensions. In the benchmark suite, every function type is used for three different dimensions resulting in a total of 45 benchmark problems. Each of these is solved 25 times and the average performance is used to measure the overall performance of the algorithms under consideration. The functions are grouped in two groups. The first group, the *extended simple functions*, consists of 8 types of functions (24 functions in total) with dimensions ranging from 2 to 20. These are extensions of commonly used multimodal problems. The second group, the *composition functions*, consist of 7 types of functions (21 function in total) with dimensions ranging

Table 4: Comparison of the proposed Algorithm 1 and the algorithms of the GECCO 2020 Competition on Niching Methods. The proposed algorithm ranks on position 5 wrt. to the score defined by scenario I. For details see [27].

algorithm	score	rank
RS-CMSA-ESII	0.9063	1
GaMeDE	0.8923	2
CMSA-ES-DIPS	0.8335	3
CMSA-ES	0.8004	4
SHGA	0.7571	
EMSO-MMO	0.6977	5
DEnrand-H	0.5819	6
FastNichingEP	0.5549	7

Algorithm 3: Contraction

Function: Contraction

Input: set of locations of global maxima S, set of generations $G = \{g_i\}_{i=0,\dots,|G|}$, where each generation $g_i = \{x_{i,j}\}_{j=1,\dots,|g_i|}$ is a set of individuals (generations may have different numbers of individuals)

Output: contraction number $\xi \in [0, \infty[$

```
Q = ∅
for k = 1, ... |G| do
    //partitioning of g_{k-1}
    P_0 = {P_0[1], ..., P_0[|P_0|]} partition of g_{k-1} in
      clusters of nearest neighbors ∀s_i ∈ S
    //partitioning of g_k
    P_1 = {P_1[1], ..., P_1[|P_1|]} partition of g_k in
      clusters of nearest neighbors ∀s_i ∈ S
    for i = 1, ..., |S| do
        //number of individuals to compare
        n = min(|P_0[i]|, |P_1[i]|)
        for j = 1, ..., n do
            //contraction per compared individual
            Q =
              Q ∪ {|P_1[i][j] − S[i]|_2 / |P_0[i][j] − S[i]|_2}
        end
    end
end
//geometric mean over all generations
ξ =geometric mean(Q)^{|G|}
return (ξ)
```

Table 5: The *extended simple functions* from the competition on real-parameter single objective optimization at CEC-2015 [17]. The functions are specified for three dimensions d, and the numbers of global and local optima are given.

#	problem type	d	#glob/loc opt.
1	Expanded two-peak trap	5	1/15
		10	1/55
		20	1/210
2	Expanded Five-Uneven-Peak Trap	2	4/21
		5	32/0
		8	256/0
3	Expanded Equal Minima	2	25/0
		3	125/0
		4	625/0
4	Expanded Decreasing Minima	5	1/15
		10	1/55
		20	1/210
5	Expanded uneven minima	2	25/0
		3	125/0
		4	625/0
6	Expanded Himmelblau's function	4	16/0
		6	64/0
		8	256/0
7	Expanded six-hump camel back	6	8/0
		10	32/0
		16	256/0
8	Modified Vincent function	2	36/0
		3	216/0
		4	1096/0

from 10 to 30. These functions are formed by combining different basic functions. Some key characteristics of the benchmark problems are given in Table 5 and Table 6 for the *extended simple functions* and the *composition functions*, respectively.

The problems are posed originally as minimization problems (as opposed to the setting in this article). The optima for all the problems are sought in the domain $[-100, 100]^d$, within a budget of $2000d\sqrt{q}$, where d denotes the dimension and q denotes the number of optima of the problem. The *extended simple functions* and the *composition functions* are evaluated differently.

For the *extended simple functions* the average peak rate (percentage of the optima found) is used, where a solution x counts as identified if the function value $|f(x) - f^*| \leq 0.1$, where $f^* = f(x^*)$ denotes the optimal value attained at x^* and $|x - x^*| \leq 0.1d$. The average peak rates achieved by the Algorithm 1 (SHGA) together with the results from [17] are given in Table 7.

For the *composition functions* a different evaluation metric is chosen. As the algorithms are not able to find many optima (if any) of the problems, the function values of the five best solutions are determined, where the positions must at least be d (dimension) apart. The values for each composite function type, dimensions and

Table 6: Benchmark suite of the *composition functions* from the competition on real-parameter single objective optimization at CEC-2015 [17]. The functions are specified for three dimensions d, and the number of global optima is given.

#	problem type	d	#optima
9	Composition function 1	10, 20, 30	5
10	Composition function 2	10, 20, 30	5
11	Composition function 3	10, 20, 30	5
12	Composition function 4	10, 20, 30	5
13	Composition function 5	10, 20, 30	5
14	Composition function 6	10, 20, 30	5
15	Composition function 7	10, 20, 30	5

Table 7: Average peak rates of the benchmark problems 1 to 8 averaged over 25 repetitions.

#	d	CDE	NCDE	R2PS2	LIPS	SHGA
1	5	0.00	1.88	0.00	8.24	7.60
	10	0.00	0.00	0.00	7.96	4.68
	20	0.00	0.00	0.00	0.04	5.08
2	2	0.56	3.84	5.00	4.80	8.24
	5	0.00	0.00	0.00	17.68	10.44
	8	0.00	0.00	0.00	23.00	22.92
3	2	4.36	20.08	20.48	19.96	18.44
	3	0.68	17.60	8.44	32.12	29.64
	4	2.00	10.40	1.30	37.92	66.48
4	5	0.00	0.08	0.00	1.48	4.80
	10	0.00	0.00	0.00	0.00	1.76
	20	0.00	0.00	0.00	0.00	3.72
5	2	3.76	16.24	18.92	20.24	17.04
	3	0.00	0.00	0.00	0.00	31.96
	4	0.00	0.00	0.00	0.00	67.96
6	4	0.00	9.64	0.00	12.12	11.72
	6	0.00	1.44	0.00	28.04	21.68
	8	0.00	0.80	0.00	55.96	39.00
7	6	0.00	0.92	0.00	7.32	3.76
	10	0.00	0.04	0.00	12.04	3.12
	16	0.00	0.00	0.00	21.32	5.64
8	2	7.44	24.00	17.52	17.04	17.28
	3	1.60	42.68	10.00	44.48	40.60
	4	6.90	36.40	4.90	88.52	79.52

Table 8: Mean of 5 best solutions per function type and algorithm (for more details, see text).

#	CDE	NCDE	R2PS2	LIPS	SHGA
9	970.6	935.9	1742.3	912.0	932.7
10	9622.0	7524.3	18231.6	4079.0	14194.8
11	1105.7	1104.3	1125.7	1102.7	1105.6
12	1623.2	1652.6	2126.5	1296.7	2213.0
13	1594.6	1597.1	150498.9	1490.2	2063.7
14	2681.2	2382.1	3010.6	1820.3	3627.7
15	1918.7	1908.2	76678.4	1742.1	2459.6

repeats are obtained:

$$7 \text{ functions types} \times 5 \text{ values} \times \\ \times 3 \text{ dimensions} \times 25 \text{ repeats.}$$

For each function type and value, the median of 3 dimensions $\times$ 25 repeats is reported, and the mean of the 5 values per function type is used for scoring the algorithms. These median values are given in Table 8 for the different algorithms and function types.

From the results given in Table 7 and 8, we derive a ranking of our algorithm and the algorithms evaluated in [17]. We use the same lines as in the article leading us to the ranks given in Table 9. For each function type 1-8, the average peak rates for each (of three different) dimension gives rise to a ranking of the algorithms. E.g. for function type 5, dimension 2, the ranking of the algorithms *CDE*, *NCDE*, *R2PS2*, *LIPS*, and *SHGA* would be 5, 4, 2, 1, 3, respectively (see Table 7). The ranking in Table 9 give the average ranking, for each function type, averaged over the three different dimensions. For the function types 9-15 the ranking is derived from the mean of the solutions (the smaller the better). In addition, Table 9 gives the sums of the rankings for the function types 1-8, for 9-15, and for 1-15; lower values mean better performance. As can be seen, *LIPS* performs best in all three groups. In the group 1-8, the proposed algorithm *SHGA* performs second best, in the group 9-15 it performs second to last and overall on place three.

3.3. The 100-Digit Challenge (CEC2019)

The 100-Digit Challenge [18] was designed similar to the original SIAM 100-Digit Challenge from 2002. A set of 10 function are given, and the unique minimum of each function is to be found. The challenge is to find the minima with an accuracy of 10 digits. For each digit one point is awarded, hence 10 points can be reached per function and 100 points in total. The latter is defined to be the score of the algorithms under assessment. The number of points obtained for each function is taken as the average of the 25 best out of 50 runs of the algorithm. The maximum number of function evaluations was not limited. Algorithms could compete in one of two groups. The primary algorithm group comprised

Table 9: Ranking per function type and algorithm and sums of rankings (for more details, see text).

#	CDE	NCDE	R2PS2	LIPS	SHGA
1	3.33	3.00	3.33	1.33	1.67
2	3.67	3.33	2.67	1.67	1.67
3	4.67	2.67	3.33	2.00	2.33
4	3.00	2.67	2.00	1.67	1.67
5	3.00	2.67	2.00	1.67	1.67
6	4.00	2.67	4.00	1.00	2.00
7	3.67	3.00	3.67	1.00	2.00
8	4.67	2.00	3.67	2.00	2.67
9	4	3	5	1	2
10	3	2	5	1	4
11	4	2	5	1	3
12	2	3	4	1	5
13	2	3	5	1	4
14	3	2	4	1	5
15	3	2	5	1	4
F1-F8	**30.00**	**22.00**	**24.67**	**12.33**	**15.67**
F9-F15	**21**	**17**	**33**	**7**	**27**
F1-F15	**51.00**	**39.00**	**57.67**	**19.33**	**42.67**

Table 10: Scores of SHGA in the 100-Digit Challenge in comparison to a number of the 20 competitors participating the challenge (for more details, see text).

Algorithm	**F1**	**F2**	**F3**	**F4**	**F5**	**F6**
CIPDE	10	10	10	10	10	10
EBOwit.[1]	10	10	10	10	10	10
SHGA	10	10	10	10	10	10
EBOwit.[2]	10	10	10	10	10	10
jDE	10	10	10	10	10	10
...	...	...	...	...	...	...
TW-PSO	0	0	1	0	3.44	10
lbestPSO	0	0	1.92	0	2.72	3.92
ABC	0	0	1.36	0	5.16	1

Algorithm	**F7**	**F8**	**F9**	**F10**	**Total score**
CIPDE	1	1.72	3	10	**75.72**
EBOwit.[1]	0.56	1.68	2.08	10	**74.32**
SHGA	1.04	0.08	2.84	10	**73.92**
EBOwit.[2]	0.28	1.12	3	10	**73.04**
jDE	1.88	1.12	2.04	8	**73.04**
...	...	...	...	...	...
TW-PSO	0	=	2	10	**26.44**
lbestPSO	0	0	2	1.16	**11.72**
ABC	0	0.08	2	0.04	**9.64**

algorithms tuned for the benchmark problem (with certain restrictions given). The secondary algorithm group comprised algorithms which were not tuned to perform better on the benchmark suite. The score reached by the primary algorithms were in the range [51.92, 100]. The scores of the secondary algorithms were in the range [9.64, 75.72], naturally lower. For details, we refer to [18]. We used SHGA without modification and compare its performance to the secondary algorithms. Our algorithm reached a total score of 73.92 and would rank place 3 among the $20+1$ algorithms. The scores of the proposed algorithm and a few competitors for each of the 10 functions are given in Table 10. For all results, we refer to [28].

3.4. Constraint optimization in climate research

The Coupled Model Intercomparison Project (CMIP) is at the heart of current investigations of possible pathways of expected future global warming. It started in 1996 with a handful of global coupled climate models (CMIP1) and in the current phase (CMIP6) has more than 50 models producing future climate projections. Despite significant progress, the inter-model uncertainty of these future projections is still relatively high, depending on the variable and region considered. Evaluating these models is challenging due to their complexity, sparsity of observations and unknown future dynamics. The emergent constraint approach, see, e.g., [29], allows for the use of present-day measurements to constrain future spread in the model ensemble. At the core of this concept lies the idea that certain present-day dynamics will influence the future projection of a model. Collectively, the systematic bias in the present-day dynamic between the models can be used to constrain the spread in the future projections through cross-correlation across the model ensemble. Climate scientists have put a lot of effort into finding such emergent constraints for certain areas and processes. In previous attempts, however, the investigated areas were often pre-defined and the effort of the investigation was largely focused on finding highly correlated dynamics. Here, we take the opposite approach, where the correlated dynamics are already determined, and we try to optimise the area of the present-day dynamics used in the emergent constraint. The correlated dynamics that we consider are the spatially averaged modelled scalar value of the anthropogenically altered future CO_2-uptake over the North Atlantic (here denoted as Y) and the present-day modelled percentage of deep ocean CO_2-storage (here denoted as X). The details go beyond the scope of this paper and we refer to [19] for further details.

Here, we describe the related mathematical optimization problem. The aim is to identify regions e that optimize the cross-correlation between Y and X(e). These regions are supposed to lie within a 3D-domain, the North Atlantic ocean between America to the west and Europe/Africa to the east, restricted to latitudes between 0° and 75°N, from the sea surface to the sea bottom, see Figure 1. The data of X is available on a longitude-latitude-depth axis-parallel grid with a grid spacing of $1^\circ \times 1^\circ \times 100m$. Only cells with a model variance larger than the spatially-averaged model ensemble standard deviation of $4.84e-08$ are considered, so that the optimal region can be used to constrain model spread. We denote

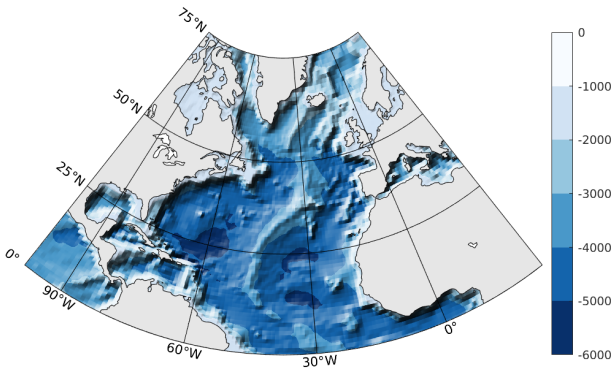

Figure 1: The area under consideration lies in the North Atlantic ocean between America to the west and Europe/Africa to the east, restricted to latitudes between 0° and 75°N, from the sea surface to the bottom.

this set of cells $\mathcal{G}$ and seek the set of cells (a subset of $\mathcal{G}$) that maximizes the correlation (in terms of the Pearson-correlation coefficient) of the randomly picked 11 models' average X and Y. In order to regularize this ill-posed problem, only areas resembling ellipsoids are considered. The set of these discrete ellipsoids $\mathcal{E} \subset 2^{\mathcal{G}}$ is defined as the image of $\gamma : E \to \mathcal{E},\ e \to \{c \in \mathcal{G} \,|\, midpoint(c) \in e\}$, where E denotes the set of all ellipsoids (subsets of $\mathbb{R}^3$). We can now define the objective function of our multimodal optimization problem. Let

$$f : \mathcal{E} \to \mathbb{R},\ e \to \rho_{X(e),Y}, \tag{5}$$

where $\rho_{X,Y}$ denotes the Pearson correlation coefficient and $X, Y \in \mathbb{R}^{11}$ is the present percentage of deep ocean CO_2 storage and future average anthropogenically altered CO_2-uptake

$$X = X(e) = (X_1(e), \ldots, X_{11}(e))$$

with

$$X_m(e) = \sum_{t,c \in e} \%\mathrm{CO_2\text{-storage(t,c) of model}}\ m|_{m=1\ldots11},$$

where t runs in monthly steps over the time period $[1999, 2009]$ (the decade surrounding the available observations normalised for the year 2004), and $m = 1 \ldots 11$ is an arbitrary enumeration of the climate models under consideration here. Correspondingly, Y is defined by its components

$$Y_m = \sum_{t,c \in \mathcal{G}} \mathrm{CO_2\text{-uptake(t,c) of model}}\ m|_{m=1\ldots11},$$

where t runs in monthly steps over the time period $[2090, 2100]$. When equipped with Steinhaus distance

$$d(e_1, e_2) := \frac{|e_1 \triangle e_2|}{|e_1 \cup e_2|},\ \forall\, e_1, e_2 \in \mathcal{E},$$

the set of discrete ellipsoids $\mathcal{E} \subset 2^{\mathcal{G}}$ becomes a finite metric space ($\triangle$ denotes the symmetric difference and $|\cdot|$ the counting measure).

In order to use Algorithm 1 to find the maxima of objective function f (5), we choose to describe an ellipsoid $e \in \mathcal{E}$ by a real-valued genome. Ellipsoids in $\mathbb{R}^3$ have 9 degrees of freedom. We choose the following representation

$$e(x, S, D) := \{y \in \mathbb{R}^3 | \\ ((y - x), e^{\pi S} D^{\dagger} e^{-\pi S}(y - x)) \leq 1\},$$

where $x \in \mathbb{R}^3$ is the center of the ellipsoid, $S \in \mathbb{R}^{3\times3}$ is an antisymmetric matrix, $D \in \mathbb{R}^{3\times3}$ is a diagonal matrix and $\dagger$ denotes the Moore-Penrose inverse. We encode (x, S, D) as $\xi \in \mathbb{R}^9$ and use the above defined map γ to define

$$\tilde{f}(\xi) : \mathbb{R}^9 \to \mathbb{R},\ \xi \to f \circ \gamma \circ e(\xi) \tag{6}$$

Due to the nature of γ, the objective function $\tilde{f}$ is piece-wise constant. The length scale of variation of $\tilde{f}$ is typically 10^{-4}, so that the function assumes likely $\geq 10^{30}$ different function values in $[0, 1]^9$. On larger scales ($\delta x \approx 10^{-2}$), the function resembles a smooth, continuous function. It turns out that it has many (local) maxima, many of them clustered together. To find all or a large part of them therefore requires an efficient multimodal optimization algorithm, such as the one presented in Algorithm 1. (Strictly speaking, for piecewise constant functions, such as $\tilde{f}$, the notion of local or global maxima is problematic. Normally, the set of local maxima equals the entire domain with the exception of a set with measure 0, and the set of strict local or global maxima is empty. Only the set of (non-strict) global maxima is non-trivial. We disregard this aspect as $\tilde{f}$ resembles a smooth, continuous function on a larger scale.)

We use Algorithm 1 with the standard configuration for a 9-dimensional search space: $n_p = 550$, $n_g = 20$ and $n_{nb} = 55$, and allow for $N = 10^6$ function evaluations. The optimization executed nine iterations of a successively increasing population (Algorithm 1, line 2) with an initial population of 550 and a final population of 4950. A total of 187 local maxima have been found. About 25% of the function evaluations were used to find the starting point for local solves (seed points), and 75% were used in local solves by the CMA-ES algorithm. We call this the *efficiency* of the multimodal optimization. Table 11 shows the statistics for the optimization run.

Because $\tilde{f}$ is piece-wise constant, the local solver (CMA-ES) identifies a number of saddle points as local maxima. Random testing of 1000 points in a 10^{-1}-Euclidean neighborhood is used to detect the saddle points (the search space is normalized to $[0, 1]^9$). After eliminating spurious maxima 170 maxima remain. The 10

Table 11: Statistics for the multimodal optimization of $\tilde{f}$ (6). Each row corresponds to an iteration of the `while`-loop (Algorithm 1, line 2). The population increases linearly by 550 in each iteration, and the number of maxima (local or global) found increases from 1 after the first iteration to 187 after the ninth iteration. The cumulative number of function evaluations is also shown. The last column shows the efficiency, the percentage of function evaluations used in local solves as part of the overall number, a measure of the efficiency of the multimodal algorithm.

Iter.	Pop. size	# Sol.	# Fct eval	Efficiency
1	550	1	10.858	41.8%
2	1,100	11	61,377	69.6%
3	1,650	25	144,715	74.4%
4	2,200	39	225,501	73.2%
5	2,750	59	335,384	73.0%
6	3,300	88	492,141	74.3%
7	3,850	115	654,946	74.4%
8	4,400	147	821,009	73.8%
9	4,950	187	997,194	73.1%

best solutions have the objective function values

$$\eta_0 = 0.9819, \quad \eta_1 = 0.9809, \quad \eta_2 = 0.9792, \quad \eta_3 = 0.9778,$$
$$\eta_4 = 0.9769, \quad \eta_5 = 0.9768, \quad \eta_6 = 0.9756, \quad \eta_7 = 0.9749,$$
$$\eta_8 = 0.9744, \quad \eta_9 = 0.9705$$

corresponding to $\xi_i, i = 0, \ldots, 9 \in \mathbb{R}^9$ ellipsoids. Since the representation of the ellipsoids in $\mathbb{R}^9$ and its metric are not natural (e.g., the map $\gamma \circ e$ is not injective, two different ξ can represent the same discrete ellipsoid), it is appropriate to investigate the maxima in the metric space $(\mathcal{E}, d)$ of discrete ellipsoids, as defined above. In order to visualize the first 10 ellipsoids in terms of their distances, a 9-dimensional Euclidean space would be necessary. If we allow for a controlled uncertainty in the display, we can display the metric relationship between the first 10 ellipsoids in the Euclidean plane, see Figure 2. In the figure, solutions are represented as circles and their corresponding distances in $(\mathcal{E}, d)$ lie in the interval of minimum and maximum distant points in corresponding circles. It can be seen that the solutions 0, 1 and 2 (and possibly 6) form one cluster, the solutions 4, 5 and 7 form another cluster, and the solutions 3, 8 and 9 are clearly separated.

A discussion of the corresponding ellipsoids in relation to a climate-related constraint optimization problem is beyond the scope of this paper. For further details, we refer to [19].

4. Discussion

The aim of this work was to design a widely applicable, easy to use algorithm for continuous, multimodal optimization problems in moderate dimensions. We chose a hybrid algorithmic approach involving (1) identifying basins of attraction and (2) carrying out local solves. For (1), we chose a standard panmictic genetic algorithm (deterministic crowding) with nearest neighbor-based detection of basins of attraction in order to avoid a priori assumptions as regards the size and shape of the basins. In order to cope with black-box objective functions, which may have an arbitrary number of maxima, the genetic algorithm is used with a successive increasing population. Local solves (2) are carried out using the standard CMA-ES solver. The key to a successful optimization algorithm is striking a proper balance between the efforts spent on global exploration and local solves. It turns out that this can be achieved by carefully choosing three parameters, two configuring the panmictic deterministic crowding algorithm (population size, number of generations) and one specifying the identification of the basins of attraction (number of individuals required per basin). A good generic choice on the base of the problem dimension seems to be possible.

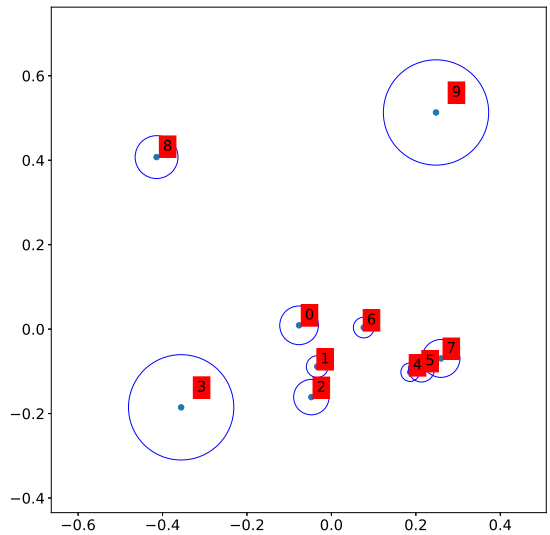

Figure 2: The ten best solutions (0 best) are represented by circles. Distances of solutions lie within the range of possible distances between the points in the circles. Solutions 0, 1 and 2 (and possibly 6) form one cluster, solutions 4, 5, and 7 form another cluster, whereas solutions 3, 8, and 9 are clearly separated.

The algorithm has been developed using the CEC2013 benchmark suite [16]. Its performance has been evaluated on that benchmark suite, on the CEC2015 benchmark [17], and the 100-Digit Challenge (CEC2019) [18]. In the average it shows medium to good performance. Better than average performance is achieved for lower dimensional problems ($d \leq 10$) and rather higher budgets, like for the first 10 functions of the CEC2013 benchmark, the first 8 function types of the CEC2015 benchmark or

the problems from the 100-Digit Challenge. On problems with higher dimensionality ($d \geq 20$) or lower budgets, like the last 10 functions of the CEC2013 benchmark or the last 7 function types of the CEC2015 benchmark the performance is below average. The requirement of a rather high budget stems from the design choice for n_p, n_g, n_{nb} as given in Table 1.

In summary, we state that our design choice is successful and that the algorithm may be valuable for a larger number of applications.

5. Conclusion

In this work we presented a two-phase multimodal optimization scheme based on the well-established deterministic crowding GA and CMA-ES. It identifies niches by making use of nearest neighbor search. The scheme is flexible enough to capture niches of various shapes and sizes. Robustness is achieved through a sensible balancing of efforts spent on exploration and exploitation. If standard choices are taken for the essential configuration parameters, the algorithm becomes an easy to use algorithm with only the stopping parameter to be specified.

Declarations

Funding, Support This work was supported by the Research Council of Norway through the project COLUMBIA (grant number 275268). The simulations were performed on resources provided by NORCE Norwegian Research Centre AS.

Conflict of interest The authors declare that they have no conflict of interest.

Human and animal rights This article does not contain any studies involving human participants or animals carried out by any of the authors.

Availability of code, data and material The software, including scripts to execute the CEC benchmarks (2013, 2015, 2019), is available from [30].

References

1. Audit C and Hare W. Derivative-Free and Blackbox Optimization. Berlin: Springer, 2017. DOI: https://doi.org/10.1007/978-3-319-68913-5
2. Bäck T, Fogel D, and Michalewicz Z. Handbook of Evolutionary Computation. Oxford: Oxford University Press, 1997. DOI: https://doi.org/10.1007/978-3-662-05094-1
3. Preuss M. Multimodal Optimization by Means of Evolutionary Algorithms. Berlin: Springer, 2015. DOI: https://doi.org/10.1007/978-3-319-07407-8
4. Mahfoud SW. Niching methods for genetic algorithms. PhD thesis. University of Illinois at Urbana-Champaign, Champaign, IL, USA, 1995
5. Mengshoel OJ and Goldberg D. The crowding approach to niching in genetic algorithms. Evolutionary Computation 2008; 16(3):315–54. DOI: https://doi.org/10.1162/evco.2008.16.3.315
6. Goldberg D and Richardson J. Genetic algorithms with sharing for multimodal function optimization. *Genetic Algorithms and Their Applications: Proceedings of the Second International Conference on Genetic Algorithms*. 1987 :41–9
7. Harik G. Finding multimodal solutions using restricted tournament selection. *Eshelman LJ (ed), Proceedings of the international conference on genetic algorithms, Morgan Kaufmann, Menlo Park*. 1995 :24–31
8. Petrowski A. A clearing procedure as a niching method for genetic algorithms. *Proc. of Third IEEE Congress on Evolutionary Computation*. 1996 :798–803
9. Engelbrecht AP. Computational Intelligence. John Wiley Sons, 2007. DOI: DOI:10.1002/9780470512517
10. Li X, Epitropakis MG, Deb K, and Engelbrecht A. Seeking Multiple Solutions: An Updated Survey on Niching Methods and Their Applications. IEEE Transactions on Evolutionary Computation 2016; 21(4):1–21. DOI: https://doi.org/10.1109/TEVC.2016.2638437
11. Goldberg D and Vössner S. Optimizing Global-Local Search Hybrids. *GECCO*. 1999
12. Moscato PA. Memetic algorithms: A short introduction. *In: Corne D, Dorigo M, Glover. F , editors. New Ideas in Optimization*. 1999 :219–34
13. Gharsalli L. Hybrid Genetic Algorithms. *In: Optimisation Algorithms and Swarm Intelligence, N. Vakhania*. 2022
14. Julstrom B. Comparing Darwinian, Baldwinian, and Lamarckian search in a genetic algorithm for the 4-cycle problem. *Genetic and Evolutionary Computation Conference, Late Breaking Papers, S. Brave and A. S. Wu, Eds. Orlando*:134–138]
15. Kronfeld M and Zell A. Towards scalability in niching methods. *Proceedings of the 2010 IEEE Congress on Evolutionary Computation (CEC'10)*. 2010 :1–8
16. Li X, Engelbrecht A, and Epitropakis M. Benchmark Functions for CEC 2013 Special Session and Competition on Niching Methods for Multimodal Function Optimization. Technical Report. Evolutionary Computation and Machine Learning Group. RMIT University, Australia, 2013 :1–10
17. Qu BY, Liang JJ, Wang ZY, Chen Q, and Suganthan PN. Novel Benchmark Functions for Continuous Multimodal Optimization with Comparative Results. Swarm and Evolutionary Computation 2016; 26:23–34. DOI: 10.1016/j.swevo.2015.07.003
18. Price KV, Awad NH, Ali MZ, and Suganthan PN. The 100-Digit Challenge: Problem Definitions and Evaluation Criteria for the 100-Digit Challenge Special Session and Competition on Single Objective Numerical Optimization. Technical Report. Nanyang Technological University, Singapore, 2018 :1–21

19. Goris N, Johannsen K, and Tjiputra J. Gulf Stream and interior western boundary volume transport as key regions to constrain the future North Atlantic Carbon Uptake. Submitted, under revision. 2022
20. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, and Duchesnay E. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 2011; 12:2825–30
21. Hansen N, Ostermeier A, and Bar F. Completely Derandomized Self-Adaptation in Evolution Strategies. IEEE Computational Intelligence Magazine 2001; 9:159–95
22. Hansen N, Akimoto Y, and Baudis P. Pycma - a Python implementation of CMA-ES. https://github.com/CMA-ES/pycma. 2019
23. Deb K and Kumar A. Real-coded Genetic Algorithms with Simulated Binary Crossover: Studies on Multimodal and Multiobjective Problems. Complex Systems 1995; 9:431–54
24. Fortin FA, Rainville FMD, Gardner MA, Parizeau M, and Gagné C. DEAP: Evolutionary Algorithms Made Easy. Journal of Machine Learning Research 2012; 13:2171–5
25. Eiben AE and Smith JE. Introduction to Evolutionary Computing. Berlin: Springer, 2015. DOI: https://doi.org/10.1007/978-3-662-05094-1
26. Butenko S, Pardalos PM, and Shylo V. Optimization Methods and Applications. Berlin: Springer, 2017. DOI: https://doi.org/10.1007/978-3-319-68640-0
27. Li X, Engelbrecht A, and Epitropakis M. GECCO 2020 Competition on Niching Methods for Multimodal Optimization. https://github.com/mikeagn/CEC2013. 2020
28. Price KV, Awad NH, Ali MZ, and Suganthan PN. The 2019 100-Digit Challenge on Real-Parameter, Single Objective Optimization: Analysis of Results. Technical Report. 2019 :1–22
29. Hall A, Cox P, Huntingford C, and Klein S. Progressing emergent constraints on future climate change. Nat. Clim. Change 2019; 9:269–78. DOI: https://doi.org/10.1038/s41558-019-0436-6
30. Johannsen K. SHGA. 2022 Oct. DOI: 10.5281/zenodo.7137341. Available from: https://doi.org/10.5281/zenodo.7137341